Entrega Final — Predicción de Churn en E-commerce (Olist)

Modelos, API, Tablero React/Vite y Despliegue en AWS EC2

David Alan Delgado Zarco Octavio Esquivel Alvarez del Castillo
John Williams Morrill Jason Rosher Morrill

Marzo 10, 2026

Abstract

Se presenta el producto final de predicción de churn para clientes del dataset Olist con ventana W=90 días. El label se construye sin data leakage usando una arquitectura de ventana temporal (snapshot + ventana futura). Se entrenan y comparan Regresión Logística y Random Forest, evaluados con PR-AUC para la clase no-churn. Los experimentos se registran en MLflow desplegado en AWS EC2. Como producto final se entrega un tablero React/Vite servido por una API Express (Node.js + Python joblib) y desplegado en EC2 con PM2, que permite estimar la probabilidad de churn de un cliente junto con la contribución interpretable de cada feature.

1 Resumen del problema

Contexto. Olist es un marketplace brasileño que conecta vendedores con clientes a través de múltiples canales. El historial de transacciones muestra que la gran mayoría de los compradores realiza una sola compra, generando un fuerte desbalance entre clientes que regresan (no-churn) y los que no vuelven (churn). Retener un comprador recurrente tiene un valor económico significativamente mayor que adquirir uno nuevo, por lo que identificar de forma temprana a los clientes en riesgo es una prioridad operativa.

Pregunta de negocio y alcance. *¿Es posible anticipar, a partir del historial transaccional y de experiencia de un cliente, si volverá a comprar en los próximos 90 días?* El alcance incluye: (i) definición rigurosa del label sin leakage, (ii) pipeline de features a nivel cliente, (iii) modelo supervisado desplegado en una API, y (iv) tablero interactivo accesible en la nube.

Datos. Se utilizaron cinco archivos del dataset público de Olist: `orders`, `customers`, `payments`, `reviews` e `items`. Cada tabla 1:N se agrega por `order_id` antes del join para evitar duplicación de registros. El dataset final contiene clientes activos con al menos una compra en un lookback de 180 días previo al snapshot.

Cambios respecto a la Entrega 2.

- El tablero migró de Streamlit a **React/Vite + API Express (Node.js)**, con predicción en tiempo real y panel de contribución de features.
- La inferencia se sirve vía `POST /api/predict` usando el modelo `.joblib` invocado desde Python como proceso hijo (`spawn`).

- El despliegue se automatizó con `deploy_churn_dashboard.sh` y se gestiona con PM2 en EC2 (puerto 3000).
- El reporte incorpora métricas con umbral ajustado ($\hat{\tau} = 0.12$) y manuales de usuario e instalación.

2 Datos y construcción del label

Construcción del label sin leakage

Se define una **fecha de snapshot** y una **ventana futura** $W = 90$ días. El churn se asigna observando compras exclusivamente en el período posterior:

$$\text{churn} = 1 \iff \text{el cliente no compra en } (\textit{snapshot},\ \textit{snapshot} + 90 \text{ días}]$$

churn = 0 (no-churn / recompra) si el cliente sí compra dentro de la ventana. La población se restringe a clientes con al menos una compra en el lookback de 180 días previo al snapshot, evitando incluir clientes históricamente inactivos. Se verificó que no existan duplicados por `order_id` (duplicados = 0), garantizando integridad en los agrupados.

3 Features y partición temporal

Features a nivel `customer_unique_id`

- **RFM:** *Recency* (días desde última compra hasta snapshot), *Frequency* (órdenes únicas), *Monetary* (suma de pagos).
- **Experiencia / logística:** promedio de review score, días de entrega, días de retraso vs. fecha estimada, número de ítems, suma de precio y suma de flete.
- **Contexto geográfico:** `customer_state` (one-hot encoding, `handle_unknown="ignore"`).

Partición temporal

Para evitar *data leakage* se ordenan los clientes por su última compra previa al snapshot y se separan por percentil 80: el 80 % más antiguo forma el conjunto de entrenamiento y el 20 % más reciente el de prueba. Esto simula un escenario de producción donde el modelo se evalúa en clientes cuyo futuro aún no se conoce.

4 Modelos y evaluación

Baseline y desbalance

El dataset presenta fuerte desbalance: la clase minoritaria es el no-churn (recompra). Un clasificador trivial que prediga siempre churn alcanza alta accuracy pero obtiene $F_1^{(0)} = 0$, lo que motiva el uso de métricas enfocadas en la clase de interés.

Modelos comparados

Se entrenaron dos pipelines con preprocesamiento idéntico (imputación mediana + escala estándar para numéricas; imputación moda + one-hot para categóricas):

- **Regresión Logística**: `class_weight=balanced`, `C=0.1`, `max_iter=2000`.
- **Random Forest**: `class_weight=balanced_subsample`, `n_estimators=400`.

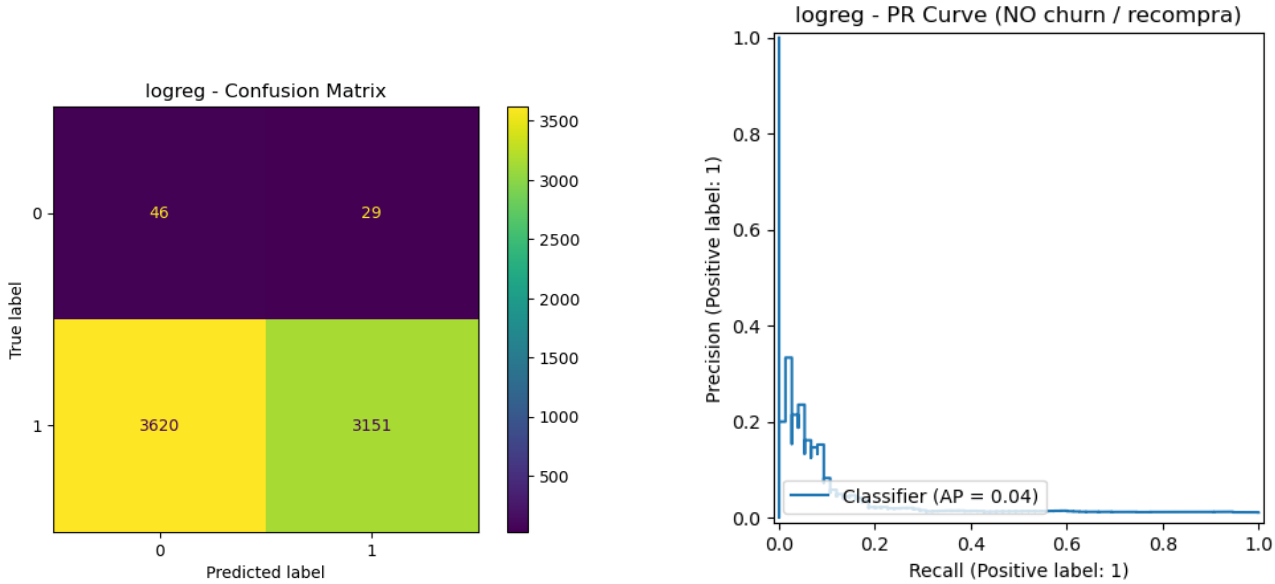
Métricas y selección de umbral

La métrica principal es el **PR-AUC para no-churn** (`pr_auc_nochurn_0`). Se realizó un barrido de 99 umbrales para la Regresión Logística, seleccionando $\hat{\tau} = 0.12$ como el valor que maximiza $F_1^{(0)}$.

Table 1: Comparación de modelos (umbral 0.5 para comparación directa; LogReg con $\hat{\tau} = 0.12$ en la última fila).

Modelo	Acc.	F1 (churn)	Prec. (no-ch.)	Rec. (no-ch.)	F1 (no-ch.)	PR-AUC (no-ch.)
Baseline (todo churn)	alto	alto	0.000	0.000	0.000	—
Random Forest	0.989	0.994	0.000	0.000	0.000	0.014
LogReg ($\tau = 0.50$)	0.467	0.633	0.013	0.613	0.025	0.035
LogReg ($\hat{\tau} = 0.12$)	—	—	—	—	**0.113**	**0.035**

La Regresión Logística supera al Random Forest en PR-AUC para no-churn (0.035 vs. 0.014) y es el único modelo que detecta la clase minoritaria de forma no trivial. El umbral ajustado a 0.12 eleva $F_1^{(0)}$ de 0.025 a 0.113. El run ganador en MLflow es `logreg_C0.1_thr012`.



(a) Matriz de confusión — LogReg ($\hat{\tau} = 0.12$).

(b) Curva PR para no-churn — LogReg.

Figure 1: Evaluación del modelo ganador.

5 Soportes en MLflow (AWS EC2)

MLflow Tracking Server se desplegó en una instancia **EC2 Ubuntu** (puerto 5000, backend SQLite, artefactos en `/home/ubuntu/mlruns`). Se documentan evidencias con IP pública y usuario. Repositorio: https://github.com/Jason-Morrill/Microproyecto_proyectogrado

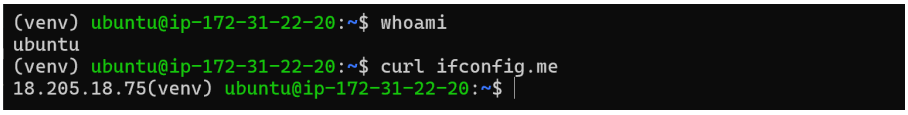
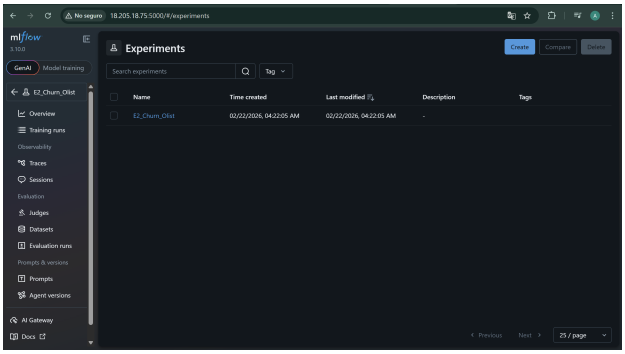
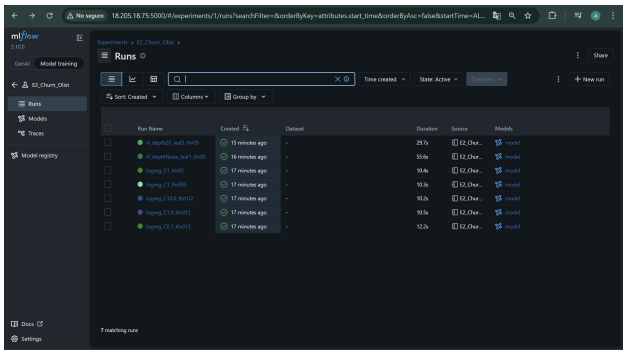


Figure 2: EC2: usuario e IP pública.

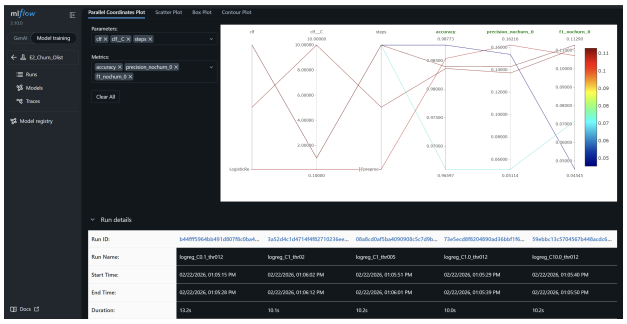


(a) UI MLflow con IP pública.

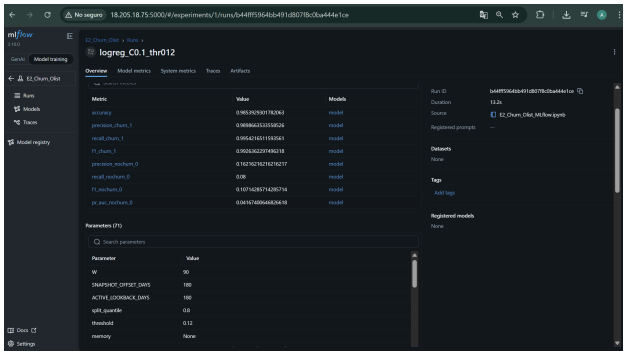


(b) Lista de runs del experimento.

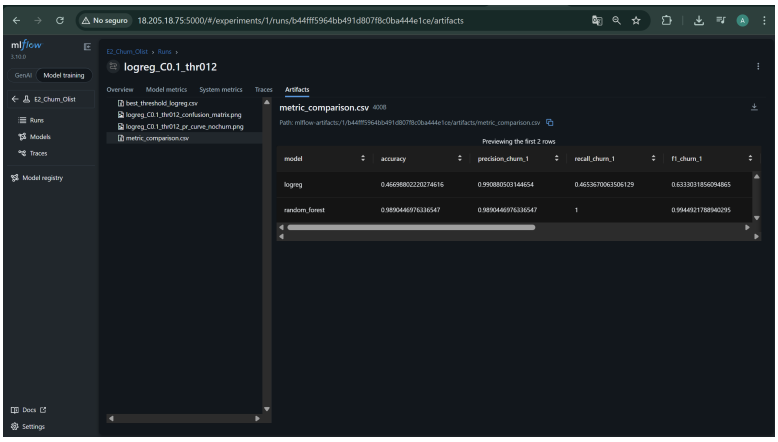
Figure 3: MLflow: acceso y experimentos.



(a) Comparación de runs (`pr_auc_nochurn_0`).



(b) Run ganador: parámetros y métricas.



(c) Run ganador: artefactos registrados.

Figure 4: MLflow: comparación y run ganador.

6 Tablero y API

Arquitectura

El producto final es un tablero de predicción individual accesible desde el navegador, desplegado en la misma instancia EC2 con PM2 en el puerto 3000. Consta de tres capas:

1. **Frontend React/Vite**: formulario lateral que captura las 10 variables del cliente y muestra el resultado.
2. **API Express (Node.js)**: endpoint `POST /api/predict` que valida el payload, invoca Python vía `spawn` y devuelve JSON.
3. **Inferencia Python**: `predict_joblib.py` carga `best_model_logreg.joblib` y ejecuta `predict_proba`, retornando `churnProbability` $\in [0, 1]$.

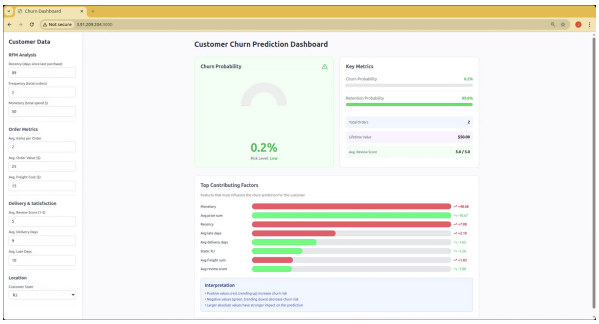
$$\text{Usuario} \xrightarrow{\texttt{POST /api/predict}} \text{Express} \xrightarrow{\texttt{spawn}} \text{Python} \rightarrow \texttt{joblib} \xrightarrow{\texttt{churnProbability}} \text{Frontend}$$

Variables de entrada

`Recency`, `Frequency`, `Monetary`, `avg_review_score`, `avg_delivery_days`, `avg_late_days`, `avg_num_items`, `avg_price_sum`, `avg_freight_sum`, `customer_state`.

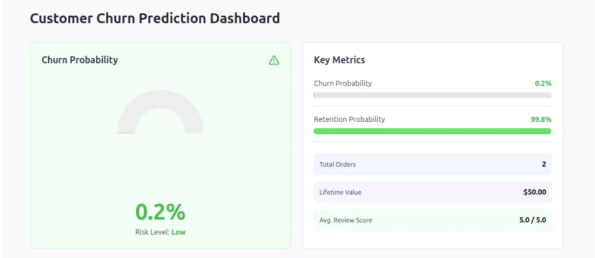
Funcionalidades del tablero

- Predicción individual: probabilidad de churn en %, nivel de riesgo (bajo / medio / alto) y probabilidad de retención complementaria.
- Panel de **contribución de features**: muestra las 8 variables con mayor impacto en la predicción, derivado directamente de los coeficientes del modelo, haciendo el resultado interpretable y accionable.
- Despliegue con `deploy_churn_dashboard.sh`: automatiza build Vite, creación de `.venv`, instalación de dependencias Python y arranque con PM2.



(a) Predicción individual con nivel de riesgo.

(b) Visualizaciones macro del tablero.



(c) Distribución de churn en la población.

Figure 5: Tablero de predicción — vistas principales.

7 Video explicativo

El video de presentación del proyecto (máx. 10 min) está disponible en el siguiente enlace de Google Drive:

https://drive.google.com/file/d/1YxZzugbef--xkANxXuvIvfrlS-3iVf4P/view?usp=sharing

El video cubre: contexto del problema, definición del label, features y partición temporal, modelos y evaluación, demostración del tablero y principales conclusiones.

8 Conclusiones y trabajo futuro

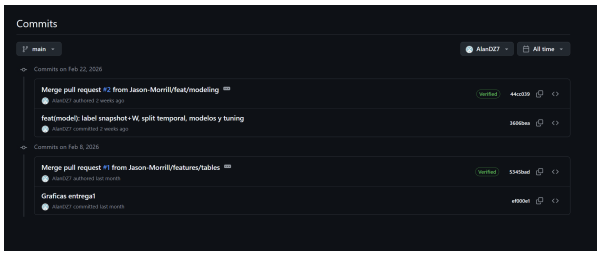
- El dataset de Olist presenta desbalance extremo (recompra rara). El accuracy convencional no es una métrica válida; PR-AUC y F_1 para no-churn son las métricas relevantes.
- La Regresión Logística supera al Random Forest en la detección de recompra (PR-AUC 0.035 vs. 0.014). Su interpretabilidad vía coeficientes permite construir el panel de contribución de features sin herramientas adicionales.
- El ajuste del umbral de decisión ($\hat{\tau} = 0.12$) incrementa $F_1^{(0)}$ de 0.025 a 0.113, demostrando que el calibrado del umbral tiene mayor impacto que la elección del algoritmo en datasets desbalanceados.
- MLflow en EC2 proporciona trazabilidad completa y reproducibilidad de los experimentos sin sobrecarga operativa significativa.
- El stack React/Vite + Express + Python + PM2 permite un despliegue reproducible y con gestión de ciclo de vida del proceso.
- **Trabajo futuro:** incorporar features por categoría de producto, explorar gradient boosting con calibración de probabilidades, y añadir monitoreo de deriva del modelo (*data drift*) en producción.

Reporte de trabajo en equipo

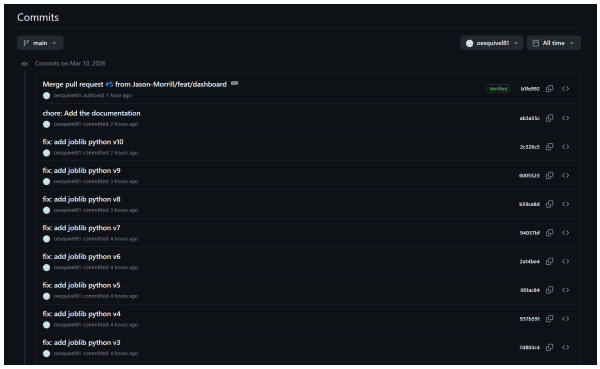
Distribución de actividades — Entrega Final

Miembro	**Rol**	**Entregables**	**Evidencia**
David Alan Delgado Zarco	Modelado y metodología	Label sin leakage, features (RFM + logística), split temporal, entrenamiento LogReg/RF, barrido de umbral, joblib	Commits en `E2_Churn_Olist_MLflow.ipynb` y `outputs/`
Octavio Esquivel Alvarez del Castillo	Reporte y QA	LaTeX entrega final ($\leq$10 pág), integración de evidencias, revisión de rúbrica, manuales de usuario e instalación	Commits en `report/`
John Williams Morrill	Tablero (React/Vite + API)	Frontend React/Vite, API Express (`server/index.js`), panel de contribución de features, script de despliegue	Commits en `Tablero_final/`
Jason Rosher Morrill	MLOps / Infra (EC2 + MLflow)	MLflow Tracking Server en EC2, runs con artefactos, PM2, capturas de evidencia AWS/MLflow	Commits en `server/` e `infra/`; capturas `evidence/aws` y `evidence/mlflow`

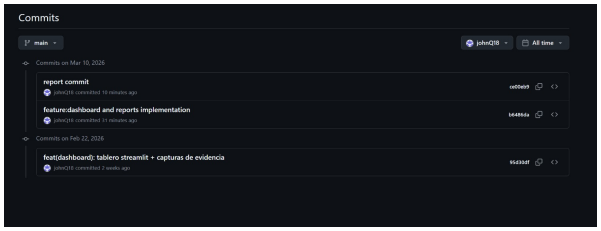
Evidencia de commits en GitHub



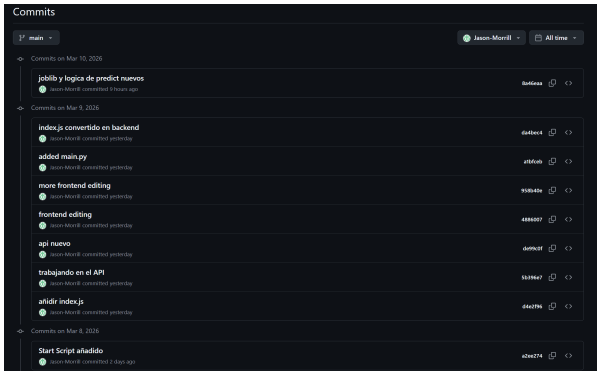
(a) Commits — David



(b) Commits — Octavio



(c) Commits — John



(d) Commits — Jason

Figure 6: Evidencia de contribuciones individuales en el repositorio.